



Test Plan

IBM SkillsBuild VR Game

Group 11

Alexandre Pinheiro Dias

cgfv65

Muhammad Rafay Abbas

djsh68

Jing Lei Wong

zlnm44

Siang Wei Law

rwbc54

Rohab Kashif

dwh45

Contents

1. Introduction	2
1.1. Provide usable access to the developed system, along with clear instructions on how to set it up and run it	2
1.2. Present the status of each behavioural requirement in a table. Provide the code (e.g., BR1.2 is “behavioural requirement for feature 1, scenario 2”) and a succinct description (no need to restate the entire user story). Indicate whether the requirement remains unchanged or has been modified (if modified, provide the updated description), and explain the extent to which it is met or not met, with justification if applicable.	3
2. Technical Development	6
2.1. Clearly describe the source materials that form the basis for the conceptualisation and development of the system.	6
2.2. Provide a clear and appropriately detailed technical description of how each system functionality was developed:	6
2.3. Clearly describe how the system’s usability and user experience aspects were addressed, providing an appropriate level of detail.	14
3. Use Instructions	15
3.1. Installation: Describe system requirements, including the minimum and recommended hardware requirements (e.g., CPU, RAM, and storage) and operating systems. Provide step-by-step instructions for installing and configuring the software.	15
3.2. Deployment: Explain how to deploy the system on a local machine and how to set up the database.	16
3.3. Launching: Provide instructions on creating a user account or logging into the system. Guide users through any first-time setup steps, such as creating admin accounts or configuring user settings.	18
3.4. Troubleshooting: List common error messages and solutions for resolving them. Explain where users can find logs or diagnostic information to troubleshoot issues.	18
4. Maintenance and Implications	19
4.1. Provide useful and usable information on how the system can be maintained	19
4.2. Provide useful and usable information on how the system’s possible future development can be implemented	19
4.3. Describe potential ethical and societal impacts of the system in its current and possible future status	19

1. Introduction

This technical report documents development, implementation, and evaluation of a web-based game (accessible through VR and in a traditional 3D style) designed to educate users about IBM SkillsBuild courses. Developed by Group 11, the project integrates educational content into an interactive, space-themed gaming experience. Below, we present the project's motivation, goals, system access instructions, and the status of its behavioural requirements.

This IBM SkillsBuild VR Game project develops an immersive VR and 3D game platform to teach high-demand technical skills such as AI, cybersecurity, and data analytics—by engaging users in navigating inside a spaceship through quiz-based challenges derived from IBM SkillsBuild content.

This gamified approach, coined by our client Mr. John Mc Namara from IBM, enhances accessibility and motivation, aligning with IBM's mission to expand technical education. The concise prototype, inspired by Paradroid and limited to 20 minutes, integrates educational content, adaptive difficulty, and a space-themed narrative to attract diverse learners while ensuring sustained engagement.

1.1. Provide usable access to the developed system, along with clear instructions on how to set it up and run it

The VR game is accessible via a web browser, utilising the Godot game engine's WebXR support to eliminate the need for local installation. To run the system, users require a compatible browser (e.g., Firefox or Chromium, latest versions) and a stable internet connection. For an immersive VR experience, a WebXR-compatible VR headset (such as Oculus Quest 2/3) is recommended, though the game also supports traditional 3D gameplay for users without VR hardware.

The game is accessible through the web on desktop and VR platforms, this eliminates the need for end user installation, reducing the barrier to trying the game.

To set up and run the game:

1. Navigate to the project's deployment URL (<https://ibm.alexdiass.dev>).
2. Ensure the browser is updated and supports WebAssembly (WASM), and additionally WebXR if wanting to play in VR.
3. For VR mode:
 - Connect a VR headset to a computer or use a standalone device with a built-in browser.
 - Open the game in the browser and click "Play (VR)", then select a course.
 - Then accept the browser's prompt to enter immersive VR mode.
 - Use joysticks to turn and move, and right controller trigger to click a button.
4. For 3D mode:
 - Open the game in the browser and click "Play (3D)", then select a course.
 - Use WASD (keyboard & mouse) or primary joystick (controller) to move, look around with mouse (keyboard & mouse) or secondary joystick (controller), click button using left click (keyboard & mouse) or using the right trigger (controller)
5. Move forwards into question room and follow the quiz-based gameplay.

The system is designed to run in a secure HTTPS environment, with certain HTTP headers set, these are required for the game to run. Detailed installation and deployment instructions are provided in Section 3.2 of this report.

For the question bank editing software, in the source code, navigate to directory `quiz_logic`, then to `question_bank`, then on the terminal, run `python3 QuestionBankTool.py` and you can edit the question banks there by creating and loading JSONs.

1.2. Present the status of each behavioural requirement in a table. Provide the code (e.g., BR1.2 is “behavioural requirement for feature 1, scenario 2”) and a succinct description (no need to restate the entire user story). Indicate whether the requirement remains unchanged or has been modified (if modified, provide the updated description), and explain the extent to which it is met or not met, with justification if applicable.

The table below outlines the status of the behavioural requirements as defined in the Requirements Document. Each requirement’s implementation status is assessed as of March 13th, 2025, reflecting the prototype’s current development stage.

Code	Description	Status	Met/Not Met	Justification
BR1.1	Look around with VR headset	Unchanged	Fully Met	Implemented using WebXR API in Godot; headset movement accurately rotates the viewport.
BR1.2	Look around with left thumbstick (VR Controller)	Unchanged	Fully Met	Controller input mapped to camera rotation, tested across Meta Quest 2 and 3.
BR1.3	Move with right thumbstick (VR controller)	Unchanged	Fully Met	Smooth movement implemented via Godot’s input system, functional in VR.
BR1.4	Teleport via VR controller trigger	Deemed unnecessary since the rooms are too small	Not Met	Teleportation is not implemented since the rooms are too small hence it is unnecessary, and likely would be more of a hinderance and source of bugs.
BR2.1	Move with WASD keys	Unchanged	Fully Met	3D mode supports standard keyboard controls, verified across platforms.
BR2.2	Look around with mouse	Unchanged	Fully Met	Mouse input adjusts camera direction smoothly in 3D mode.
BR2.3	Teleport with right-click	Deemed unnecessary since the rooms are too small	Not Met	Teleportation is not implemented since the rooms are too small hence it is unnecessary, and likely would be more of a hinderance and source of bugs.
BR3.1	VR prompt with headset	Unchanged	Fully Met	Browser prompt to enter VR is shown after selecting a course.
BR3.2	Prompt to play 3D without headset	Unchanged	Not Met	Non-VR users will not receive a suggestion prompt to play in 3D.
BR3.3	WASM support check	Non-priority, WASM is enabled by default in all modern browsers.	Partially Met	Detects WASM support, message is only logged to browser console.

Code	Description	Status	Met/Not Met	Justification
BR4.1	Auto-save progress	Non-priority, sessions do not take long.	Not Met	Progress will be reset after restarting the game (leader-board and settings are saved).
BR4.2	Pause on browser focus loss	Unchanged	Not Met	Game will pause and save when tab loses focus (Pending).
BR4.3	Pause with menu button	Unchanged	Fully Met	Manual pause implemented with controller/keyboard input.
BR4.4	Resume from pause	Unchanged	Fully Met	Game resumes seamlessly from pause state.
BR4.5	Resume from save	Unchanged	Not Met	Game saves are not implemented, so this isn't as well
BR5.1	Session completes in 15 min	Modified: Adjusted to 10-20 min	Fully Met	Sessions are be timed to fit within 20 minutes (1min per question).
BR5.2	Notify session length	Unchanged	Not Met	Game does not display estimated completion time (pending).
BR5.3	Continue after session	Unchanged	Fully Met	Returns to main menu after completion, where users can play again
BR6.1	Select and enter course	Unchanged	Fully Met	Course selection shows available SkillsBuild courses and enters upon clicking.
BR6.2	Return to course menu	Unchanged	Fully Met	Accessible from pause and from the main menu.
BR6.3	Show course details	Unchanged	Not Met	Descriptions of courses are not shown (pending).
BR6.4	Recommend related courses	Won't be implemented	Not Met	Feature planned but will not implemented since there are only a limited amount courses.
BR7.1	Review incorrect answers	Unchanged	Fully Met	Post-session review shows questions with their given and correct answers.
BR7.2	Compare performance trends	Unchanged	Partially Met	Basic performance trends through graphs are not presented, alternatively leader-board is provided.
BR7.3	Review time per question	Unchanged	Not Met	Time taken per question is recorded but not analysed or displayed.
BR8.1	Points for correct answers	Modified score factors: correctness, time taken, question difficulty	Fully Met	Scoring system awards points accurately based on time taken, question difficulty and percentage of answer correctness (for multiple choice)

Code	Description	Status	Met/Not Met	Justification
BR8.2	No points for incorrect	Unchanged	Fully Met	Incorrect answers do not increment score.
BR8.3	Bonus for fast completion	Modified: time factor now included in question score	Fully Met	Completing individual questions fast awards more points.
BR9.1	Display high score	Unchanged	Fully Met	Leaderboard is provided and sorted by score, and displays score, time taken and session name.
BR9.2	View high score list	Unchanged	Fully Met	Leaderboard accessible from main menu.
BR9.3	Handle tied scores	Unchanged	Fully Met	Score now includes time factor, therefore equal scores are marked the same
BR10.1 and BR10.2	Question difficulty changes based on user performance	Unchanged	Fully Met	Users have a local difficulty score, which is incremented/decremented when a user answers a question, based on: time taken, the question difficulty, and whether the question is correct. This is capped at a maximum change of 0.2 per question.
BR10.3	Maintain difficulty for average	Unchanged	Fully Met	Stable difficulty for mixed performance.
BR11.1	Visual accessibility options	Unchanged	Not Met	Colourblind mode and text scaling not implemented yet.
BR11.2	Auditory accessibility options	Audio cues and subtitles are not needed since there are only sound effects.	Fully Met	Volume controls are implemented, other auditory accessibility options weren't added since current sounds are only interaction sounds which have corresponding visual cues.
BR11.3	Motor accessibility options	Unchanged	Not Met	Basic control remapping and sensitivity adjustments pending.
BR12.1	Set up local multiplayer	Unchanged	Not Met	Optional feature not implemented due to focus on core gameplay.

All critical requirements have been fully met within this prototype, partial implementations reflect ongoing efforts, while some unmet features were postponed due to prioritisation of core functionality. Where there have been adjustments to the initial requirements, justification on why has been provided.

2. Technical Development

2.1. Clearly describe the source materials that form the basis for the conceptualisation and development of the system.

In the initial project specification provided by IBM, we were given the base ideas needed to make the game. Which is a space-themed VR Quiz Game that promotes IBM SkillsBuild to the players, which also has an adaptive difficulty based on the users' gameplay.

Using the specification, as well as the initial contact (emails) and meetings (virtual), our team, alongside our practical instructor, constructed a short description of our minimum viable product (MVP) to be delivered to the client during the handover:

“A space themed vr quiz game that works with adaptive difficulty targeting IBM users using IBM SkillsBuild Platform “

This formed the basis of our development, as well as giving us a goal to work towards.

No initial source code or database was given to us, so we had to create everything from scratch.

Client asked us to use the courses within IBM SkillsBuild to source our questions to display on the game. We initially tried asking for a JSON or a database of the questions and answers used within the SkillsBuild website, which was denied due to security reasons, as well as the client wanting the team to source the questions by doing the SkillsBuild courses ourselves, so this added an additional hurdle in our game design.

2.2. Provide a clear and appropriately detailed technical description of how each system functionality was developed:

System Architecture (first draft)

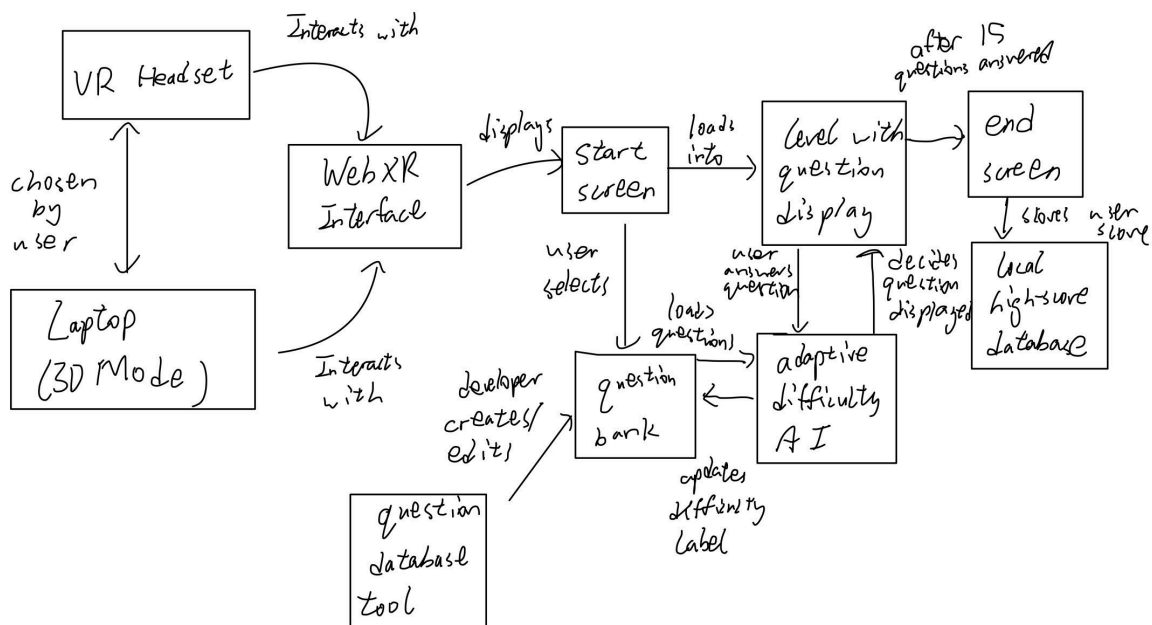


Figure 1: first draft

System Architecture (final)

This flow diagram shows the full lifetime of the final product, dotted lines represent tasks that occur in the background. Blue lines correspond to actions occurring only for the 3D viewport, and orange lines correspond to actions only occurring for the VR viewport.

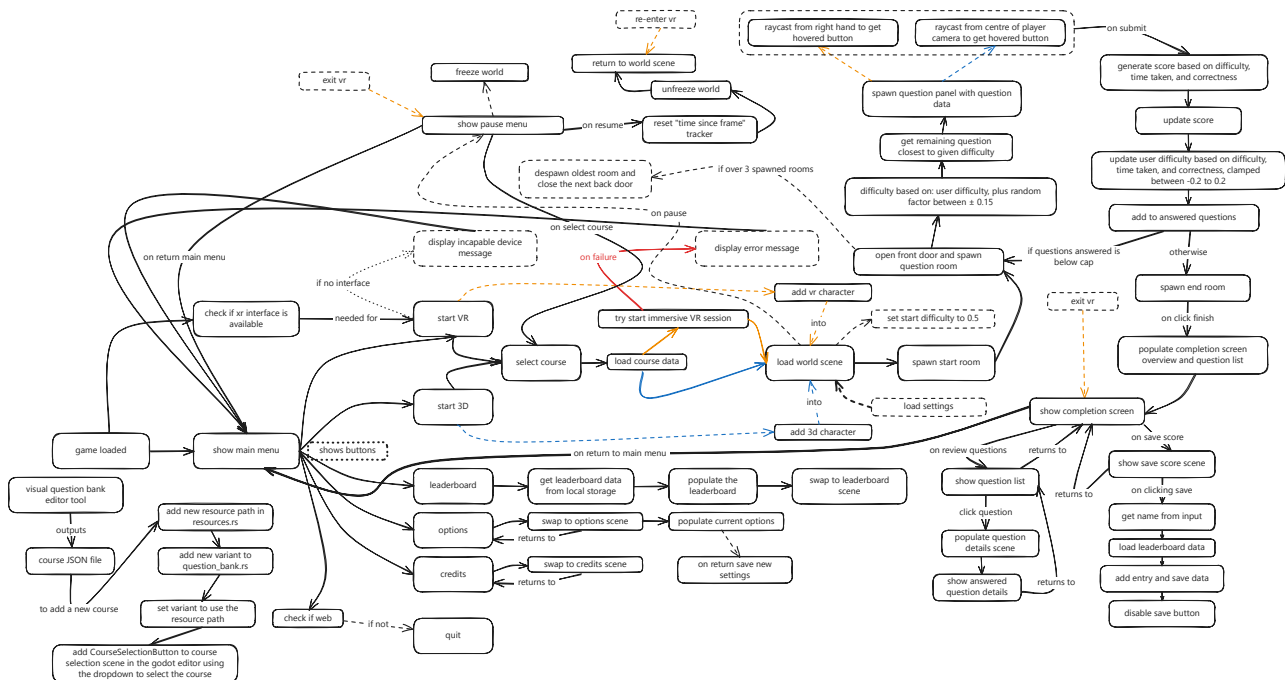


Figure 2: final architecture

Design Principles

1. Accessibility

- Besides the VR aspect of the game, made the game accessible through traditional 3D viewport, allowing users even without a VR headset to play and enjoy the game utilising a keyboard and mouse, or alternatively even a controller.
- Drafted an example for an options menu that can cater to the colourblind, as well as adjusting font size.
- Implemented adaptive difficulty for the selection of quiz questions in order to cater difficulty to all types of players, no matter their experience in the topic.

2. Extensibility

- Created question bank tool to allow the client to easily create and modify question banks for the game.
- Written code that encourages the future expansion of the game with new question banks (diagram provided in bottom left of Figure 2)
- Data about session completion is stored (which could be sent somewhere) which would allow for further analysis of questions, which could be used to make informed modification to question difficulties within the question banks.

3. Simplicity

- During discussion phase, scope of product kept getting larger and larger, team realised this issue and dealt with it by focusing on creating a base product, then building from there.
- Promoted building simple framework before adding more features on top, ensuring solid foundation before anything new is added.
- Game logic is split into reusable components, only using shared state when necessary or when it is guaranteed to be part of the session (i.e. time elapsed in world). This reduces the chances of race conditions and helps keep errors isolated in their components.

Technologies Used

For the game engine, we chose to use Godot due to the open source nature, extensive documentation and it being a relatively lightweight development environment. Using Godot allowed us to deploy to the web, which other engines have limited support for, increasing the accessibility and reducing the barrier of entry to playing the game.

Natively, Godot has its own language GDScript which is similar to python, however it is interpreted and non statically typed. Hence we chose to use Rust through Godot's GDExtension API, utilising the excellent [godot-rust](#) library. This gave us the confidence to ensure that we have defined everything correctly and will not run into runtime errors that may be hard to produce. Rust being compiled (to wasm) and statically typed, also allowed us to reduce the overhead that our code has compared to GDScript since it is already compiled, and also the bundle size of the code.

We utilised python to create the question bank tool, which is not a part of the game itself, but acts as an easy tool to create and edit course question banks, each course is stored in its own JSON file, which then can be read by the game.

Git was used for version control, using GitHub as a repository, as well as an issue tracker and kanban board. Typst was used for all external documentation outside of code, with comments left within code where needed.

Development Process

Whilst gathering our initial requirement for the project, we decided on using **Scrum** as our development approach. During actual production, we attempted and succeeded in applying this approach to our development.

Sprints were one week long, and they start at the end of our software engineering practical on Wednesdays 11am, and end at the beginning of the next practical at 9am, with the time in between spent doing retrospectives between the team members, as well as deciding each other's tasks for the week.

Tasks assigned for each other during the week is not always technical in nature. Although a good portion of it is coding game systems and menus, some time and man-hours were also spent on creating question banks, finding sound effects, textures, as well as typing up our documentation such as the testing plans and reports, showcasing all facets of the software development cycle.

On Fridays at 5pm, our team has a standup in which our team members have an allotted time to develop together and update each other on our progress. This assisted a lot in development due to certain sections of the game requiring integrating systems that two separate team members have made. Having an allotted time slot in which team members work on the product together also ensures that they are all on task and can finish their tasks by the end of the sprint. This also allowed for clarification on ongoing tasks, allowing us to keep track of deadlines and progress being made.

System Functionality 1: Scene manager

Base Architecture

This is at the core of the application, it handles the primary state of the game such as input method, course selection, initial difficulty, and xr interfaces.

This class handles the major transitions, such as between the title screen and the world, and vice versa, and dealing with setting up VR/3D. It is also a gateway to get root nodes such as the sound effects class, providing safe fallible ways to get them from a generic base pointer of any node in the scene tree.

This architecture reduces duplication of code since all handling/fetching of the major nodes is done through the scene manager, which is easily obtainable, and also helps to ensure node paths are all correct since they are done from a central location.

The scene manager also handles resuming/pausing the game, by providing functions that apply the relevant actions, which can be easily called from elsewhere.

System Functionality 2: Overarching scene classes

Base Architecture

There are several main states of the program, which are the: title screen, world scene, and completion screen.

These all require some state, such as which screen is shown, or shared data required throughout all child scenes. To make these interactions easier, each overarching scene implements its own custom class, which provide functions to do state transitions within themselves, such as transitioning to another submenu (e.g. course selection), or dealing with spawning a new question room after a question has been answered.

Where appropriate, these scenes implement custom constructors requiring the necessary data (such as completion information), ensuring that they are always initialised correctly.

There is a global helper button class for returning to a different subscene, used throughout the game, it allows assigning a return node from within the editor, which it then returns to (and hides the other relevant nodes within the screen).

Transitions between these main states are delegated to the scene manager.

Title screen

This provides some helper utilities that help swap between different sub screens of the title screen, such as the main menu, course selection, message box, leaderboard, options and credits.

World

The world scene has a custom constructor which requires a selected course, it then uses this to construct a question bank, and stores it within its state. It then setups the world as needed, by creating the start room and an initial question room, and then provides a callback function when a question is answered to add a new question room, or spawn the end room.

Completion screen

This has a custom constructor that requires a list of the answered questions, this is then used to populate the respective menus with data. This also provides transition functions between the different subscreens (question list, question details, and save score).

System Functionality 3: Question room & panel

Base Architecture

The question room class handles spawning in a room, with a question panel at its core. It provides some helper functions to open/close the front and back door, preventing users from jumping off the spaceship.

It uses a custom constructor which takes in an instance of the `Question` struct, along with the current time, which is passed to the question panel.

Question Panel

The question panel is a core component of the system, it is a custom class, which creates a 3D scene, dynamically adding buttons and text based on the provided `Question`. These buttons are all set to a collision layer of 8, allowing them to be collided against by the relevant `RayCast3D` from the player.

The creation of the question panel is divided into reusable functions, such as create button, create label, create body, etc. This allows us to reuse the common functionality e.g. for the choice buttons, and the submit button.

It also provides some functions for working out if an object provided by a `RayCast3D` is one of the buttons on this question panel, which it then uses to highlight the hovered selection, select (if clicked), or submit, these are split into further functions making it easier to follow.

On submission, the button on the question panel are disabled, and it calls `question_answered` on the world scene with the answered question, which handles further state transitions (such as open front door, and spawn next question panel).

System Functionality 4: VR Environment and Movement

Base Architecture

When the game is loaded, utilising Godot's `XrServer` singletons, we probe for VR support, which responds with a signal, allowing us to listen and enable playing in VR if it is supported.

When a user tries to play in VR, we fetch an `XrInterface`, which we use to configure the HMD, we then listen to signals provided by the `XrServer` to tell if initialising VR has succeeded (with the configuration) or failed, and also triggers signals telling us when the user has entered and exited VR. These signals are used to know when to enter the world, pause, or show a corresponding message.

All of these signals are listened to within the scene manager, which handles the relevant transitions (i.e. into the world).

Our VR character has its own subscene, which is injected into the world when the world is created and played in VR.

The root node of the subscene is a custom class which inherits from Godot's `CharacterBody3D`, allowing us to take advantage of Godot's physics engine to deal with collisions (i.e. with the floor). This class then listens for several signals, such as `physics_process`, which runs everytime physics are calculated. Within this signal we process any provided user input (such as joystick movement) and move the character correspondingly, and also trigger a raycast from the user's primary controller, which we use to detect when buttons with the game world are hovered, and likewise selected.

Within the subscene we have several primitive Godot nodes, used for VR, which are a `XrOrigin3D`, `XrCamera3D`, and `XrController3D`. The origin provides the centre of the character in VR, whilst the camera provides the viewport for the HMD. The controller nodes show the position of the controllers within the game, and are used to poll for input events on them.

Movement System

Input is handled through VR controllers controllers via `XrController3D` nodes, with the right thumbstick used for directional movement and left thumbstick for horizontal rotation. The "A" button on the right controller is used for jumping, and the right trigger was used for interactions within the scene such as selecting an answer.

This input handling differs from the normal Godot paradigm so input events are fetched directly from the controller nodes, instead of the global `Input` singleton.

Movement direction is derived from main player basis, which we multiply by a the direction of the thumbstick, and then normalise. This then is converted into a vector with only horizontal values, where we then apply a speed modifier. We then check if the jump button is pressed and if the user is currently on a floor, then if so, we apply a vertical impulse to the velocity. Otherwise we take the current vertical velocity and reduce it in accordance with a defined gravity constant (9.8 m/s^2) and the time elapsed.

This is then applied using Godot's `move_and_slide` primitive which then tries to move the character sliding after a collision (such as with a wall).

Rotation System

Similar to the movement system, we fetch the left thumbstick values for the VR controller. Using this we apply scaling to reduce the sensitivity, and then apply the rotation to the character body, only to the Y plane (since users can look up using their heads). The system directly modifies character body rotation whilst maintaining HMD-independent view through the `XrCamera3D` node.

Interaction System

For interacting with objects, the right controller has a `RayCast3D` mounted to its tip, which interacts only with collision layer 8 (where all buttons are located in), this allows us to poll for collisions on the `RayCast3D`, and relate the collision with any of the active buttons in the world. When a button is found, a corresponding function is applied on the relevant target class, such as the question panel.

System Functionality 5: 3D Environment and Movement

A traditional first-person movement system implemented through Rust/Godot integration with these components:

Base Architecture:

Our 3D character also has its own subscene, which is injected into the world when the world is created and played in 3D.

The root node of the subscene is a custom class which inherits from Godot's `CharacterBody3D`, allowing us to take advantage of Godot's physics engine to deal with collisions (i.e. with the floor). This class then listens for several signals, such as `physics_process`, which runs everytime physics are calculated. Within this signal we process any provided user input (such as keyboard input and joystick movement) and move the character correspondingly.

Unlike in VR, a raycast is triggered whenever there is any user input, this provides more responsive feedback. We utilise the raycast in the same manner to detect when buttons with the game world are hovered, and likewise selected.

Within the subscene we have use a primitive Godot node for the camera (`Camera3D`), and provide simple meshes for the player body, along with a reticle in the centre of the screen to help users know what they are selecting.

Unlike the VR scene, this is able to use Godot's input system which allows us to easily remap buttons to actions, and likewise isn't dependent on nodes (such as `XrController3D`) to get input from.

Movement System:

Input is handled by using WASD keyboard input utilising the Godot `Input` singleton to get directional movement based of our action map, spacebar for jumping action ("jump" input action), and left clicking the mouse for interactions such as selecting a question choice. Also within web build, this class handles recapturing the mouse on clicking back into the game.

Movement direction is derived from main player basis, which we multiply by a the direction of the input (from keyboard WASD, or controller joystick), and then normalise. This then is converted into a vector with only horizontal values, where we then apply a speed modifier. We then check if the jump button is pressed and if the user is currently on a floor, then if so, we apply a vertical impulse to the velocity. Otherwise we take the current vertical velocity and reduce it in accordance with a defined gravity constant (9.8 m/s^2) and the time elapsed.

This is then applied using Godot's `move_and_slide` primitive which then tries to move the character sliding after a collision (such as with a wall).

Since these affect the actual player body, they are listened to within the `physics_process` signal.

Rotation System: Since we can listen to input events properly within 3D, and rotation does not effect the player position, we can handle camera rotation from the `on_unhandled_input` signal. For this, we get the current relative rotation of the player base, and apply only the horizontal movement

to it (after applying a sensitivity multiplier). Which also rotates its children (including the head/camera), this is also what the movement direction is based on.

For vertical rotation, we get the player **Head** node, and get the current rotation, applying the vertical relative movement (with a sensitivity multiplier). This is then clamped to a maximum of $\pm \frac{\pi}{2} (\pm 90^\circ)$ to prevent users from rotating their heads too far (leading to an upside down viewport). This is then applied to the **Head** node.

Interaction System:

For interacting with objects, the right controller has a **RayCast3D** mounted to the head of the player body, which interacts only with collision layer 8 (where all buttons are located in), this allows us to poll for collisions on the **RayCast3D**, and relate the collision with any of the active buttons in the world. When a button is found, a corresponding function is applied on the relevant target class, such as the question panel.

System Functionality 6: Question Bank Tooling

A desktop GUI application for managing quiz question bank, implemented with Python/Tkinter, converting the provided questions and answers into a correct JSON file usable by the quiz system.

Base Architecture:

Tool uses Tkinter framework for cross-platform GUI development using JSON-based storage system, it allows for loading an existing question bank, and creating a new one. It provides an interface to add questions, and select the right answers, and set the question difficulty. After which it can be exported into a JSON file.

JSON Operations:

The tool outputs the question bank data into a JSON file with the following format, which is deserializable by the game code: an array of question objects, where each question object has a: question, list of choices, indices of the correct choices, and the difficulty of the question.

```
[{"question": "some question", "choices": ["choice one", "choice 2"], "answers": [0], "difficulty": 0.1}, {...}]
```

User Interface Components:

Main window contains a Listbox for question preview (question text only), action buttons with vertical flow layout and a label for the output file path.

When editing, there is a toplevel window for detailed editing, which uses a grid-layout form with 7 entry fields (QuestionType, Question, Choice1, Choice2, Choice3, Choice4, Answer, Difficulty) as well as a unified save handler for create/update

System Functionality 7: Adaptive Quiz Difficulty

Base Architecture:

Adaptive difficulty is implemented through our question bank struct, this struct loads a provided course's question data, and deserialises it into **Question** structs. The question bank struct also handles keeping track of the current score, answered questions, user difficulty, and overall time taken.

The question bank then provides public functions for getting a question and submitting an answer, along with two internal functions which are used to modify the user difficulty, and score upon receiving an answer submission.

The question bank expects a course, initial difficulty mode, question limit (i.e. how many questions to provide before ending), and the initial time it was created at. We currently provide 3 initial preset difficulties (Easy - 0.3, Normal - 0.5, Hard - 0.7), which *currently* defaults to the Normal mode.

Loading course data

Course data is loaded using Godot's `FileAccess` singleton, where we read the entire course data into a buffer, and deserialise it utilising the `serde_json` library.

Dynamic Difficulty System:

The question is selected by first taking the user's current difficulty level, applying a $\pm 15\%$ random jitter on it, then finding a question with difficulty closest to that specific level, ensuring some random elements while ensuring appropriate difficulty for user.

When question is answered correctly/wrongly, the user's current difficulty is adjusted based on time taken and question difficulty, this is capped at a limit of ± 0.2 ensuring that the difficulty does not change dramatically.

The answered question is also stored (and could be sent out), allowing the client to adjust the difficulty of the questions based on how often users fail it, ensuring that the question's difficulty is not arbitrarily set and instead is based on users answering the questions.

Performance Tracking:

For scoring, the correctness of answer, the time taken for user to answer, as well as the difficulty of the questions is taken into account, providing either 0 points, for an incorrect answer, or up to a limit of 250 points for a correct question (with a minimum of 10). To score highly, users must answer difficult questions fast. This information is shown within the HUD.

System Functionality 8: Leaderboard

Base Architecture:

The leaderboard scene has its own class, which updates it's contents when loaded with data from a leaderboard JSON file within the user data path (i.e. cookies and local storage on web). If this file doesn't exist it provided an empty leaderboard with a message stating the leaderboard is empty.

The leaderboard entries are stored as an array of JSON objects, and similar to the rest of the code, are serialised and deserialised utilising `serde_json`. Each entry of the leaderboard has the following information: name (associated with the entry), score, and time taken (in milliseconds) to complete the entire session.

```
[{"name": "Fastest AI", "score": 1040, "time_taken": 2023440}]
```

The entries are sorted in a descending order, ensuring that the top scorers are displayed first, with a random tie breaker for even scores. Since scores also take into account difficulty of questions, and time taken it provides a useful metric for which sessions were the best.

Submitting an entry

Within the completion screen after finishing a course, an button is shown allowing users to save their score, this leads to a subscene with a text box to enter a display name for the leaderboard, and a button to save it, after clicking the button, the leaderboard file is read (or created if it doesn't exist), the entry is appended to the end of it, and then subsequently written back into the leaderboard file. After submission the save button is disabled to prevent duplicate submissions in the leaderboard

The Role of Behavioural-Driven Development in Implementation Phase

The Behavior-Driven Development (BDD) methodology played a critical role in ensuring that the technical implementation aligned with the project's objectives during the implementation phase. Key contributions included:

1. Feature Isolation

BDD's focus on vertical slices enabled parallel development of:

- VR and 3D Interaction Layer: Movement in game and question displays in room

- Adaptive Difficulty Backend: Difficulty jitter ($\pm 15\%$), nearest-match selection and constantly changing user and question difficulties
- Data Pipeline: Python Question Bank Tool -> JSON -> Reading the data bank from the rust code

2. Iterative Validation

BDD cycles enabled:

- Early feedback on VR comfort (movement sensitivity thresholds)
- Progressive refinement of adaptive algorithms based on test-runner metrics
- Notice to change CSV files to JSON files for better parsing (We used CSV initially but chose to change to JSON to have backend interpret it better)

BDD ensured the final system met both technical correctness and educational validity (adaptive pacing matching cognitive load theories), while maintaining traceability between stakeholder requirements and code commits.

2.3. Clearly describe how the system's usability and user experience aspects were addressed, providing an appropriate level of detail.

Concern 1: User might not have VR headset, limiting access to playing the game

Our team has also developed a 3D mode alongside the VR mode of the game to allow even people without a VR headset to be able to try out the game and experience it, although the game was still made for VR in mind, having a 3D mode expands the potential user base, as well as enhance the user experience of those who might experience motion sickness in a VR environment.

Concern 2: System requirements might be too high for some users

It is an issue within the VR game development scene that these games have relatively high system requirements compared to their 3D counterparts, due to the increased amount of rendering needed in a VR environment. Due to this, we limited ourselves in developing the game to be played on the web. This also limits how graphically intensive the game can be due to browser limitations, as such, we limited ourselves to low poly textures and simple geometry within the game environment. As such, the game should be able to be ran on any modern system (no GPU necessarily required).

The web also allows for users to play the game without having to have install permission on their device.

Concern 3: Difficulty might be too challenging or easy for user

Users may find that questions do not vary a lot in difficulty and they may not be able to solve all of the questions, or they might just find the questions too easy, limiting user enjoyment and learning.

This was addressed by implementing an adaptive difficulty algorithm in the game. Instead of randomly choosing questions from a question bank, each question is assigned a difficulty, and users gain or lose difficulty rating based on answering questions correctly or incorrectly, while the individual question difficulty could be adapted based off data from user completions, ensuring that appropriately difficulty questions are given to the player, giving users a personalised experience with appropriate difficulty.

Concern 4: Question bank needs to be maintainable in order to add/edit/remove questions

To facilitate further learning, question banks need to be added or updated in order for more questions and topics to be selected by the user for the game. As such, our team made a developer-side tool that allows for creating and editing question banks, which then can be selected by the user in game, allowing for seamless updating of question banks. See the bottom left of Figure 2 on how to add a new question bank.

3. Use Instructions

3.1. Installation: Describe system requirements, including the minimum and recommended hardware requirements (e.g., CPU, RAM, and storage) and operating systems. Provide step-by-step instructions for installing and configuring the software.

System Requirements:

1. Hardware Requirements

Component	Minimum Requirements
VR Headset	Meta Quest (2, 3, Pro) (Optional for 3D Play)
CPU	Intel Core i5-7500 / AMD Ryzen 5 1600 or better
GPU	Integrated graphics (e.g., Intel UHD 620 or newer, AMD Vega 8)
RAM	8GB or more
Storage	5GB (SSD Preferred)

Settings aim for 30-45 frame per second within both VR and 3D

Component	Recommended Requirements
VR Headset	Meta Quest (2, 3, Pro) (Optional for 3D Play)
CPU	Intel Core i7-7500 / AMD Ryzen 7 1600 or better
GPU	NVIDIA GTX 1060 / AMD RX 580 or better
RAM	16GB or more
Storage	5GB (SSD Preferred)

Settings aim for 60+ frames per second to reduce induced motion sickness.

2. Software Requirements

Category	Requirements
Operating System	Windows 10/11, Linux
Browser	Must support WebXR if wanting to play VR e.g: Chrome

3. Installation Instructions:

1. Open a Supported Browser:

- Use a browser that supports **WebXR** to be able to access VR for the best experience.
- Browser must support WASM (enabled by default in all modern browsers).
- Recommended browsers:
 - **Chromium based: Chrome, Brave, other up to date derivatives** (desktop platforms)
 - **Builtin browser: Quest browser** (for VR devices)
 - **Mozilla Firefox** (not recommended due to issues with cursor)

2. Go to the Game URL:

- Open a new tab in your browser.
- Access the game URL through the browser. (<https://ibm.alexdiass.dev>)

3. Wait for the Game to Load:

- The game will start loading automatically.
- Ensure you have a stable internet connection whilst loading the assets.

4. Launch the Game:

- Once the game loads, choose to either play it in **3D** or **VR**
- Alternatively, you may choose to select **Leaderboard**, **Options**, or view the **credits**.
- Accept any prompts shown by the browser, such as entering VR.

3.2. Deployment: Explain how to deploy the system on a local machine and how to set up the database.

3.2.1 Automatic Installation of Environment:

Windows:

1. Install Git:

- Git is required for cloning the repository and managing version control.
- Download and install Git from the official website:
`https://git-scm.com/downloads`
- During installation, ensure you select the option to add Git to your system PATH.

2. Install Just (Command Runner):

- Just simplifies the process of running commands and managing dependencies.
- Install Just using Winget (Windows Package Manager):
`winget install -e --id Casey.Just`
- If you choose not to use Just, you will need to manually install dependencies, set up environment variables, and configure the project.

3. Open a Terminal:

- Navigate to the root project directory (the folder containing the project structure).
- You can open a terminal by:
 - Opening File Explorer, navigating to the project folder, clicking the address bar, and typing `wt` (Windows Terminal) or `cmd` (Command Prompt).

4. Install Dependencies:

- Run the following command to install all required dependencies:
`just setup`
- Follow any instructions if they pop up (such as installing windows c++ build tools)
- If the setup is successful, you will see a confirmation message.

5. Ensure you compile the rust code

- Run `just rust-debug rust-release`, to compile the rust GDExtension. Without this step it will fail to launch the game.

Linux & Mac:

1. Ensure Your System Packages are Installed and Up To Date:

- Ensure the following system packages are up to date with the latest versions to avoid compatibility issues (they should be available in your package manager):
 - `git`
 - `just` (minimum version v1.38)
 - `clang`
 - `unzip`
 - `python`
- If the aforementioned packages are not installed, they can be installed using the following command in the terminal on debian based distributions:


```
sudo apt install git just python clang unzip
```

- If your package manager does not have the latest version of `just` available, download it from the GitHub release and place at `~/.local/bin/just` (Linux) or `/usr/local/bin/just` (Mac).

2. Open a Terminal:

- Navigate to the root project directory using the terminal.

3. Install Dependencies:

- Run the following command to install all required dependencies:

```
just setup
```

- If the setup is successful, you will see a confirmation message.

4. Ensure you compile the rust code

- Run `just rust-debug rust-release`, to compile the rust GDExtension. Without this step it will fail to launch the game.

3.2.2 Manual Installation of Environment:

In the case of a user wishing to manually install the environment, you'll have to make sure these pre-requisite dependencies are installed:

- **emsdk + emscripten v3.1.74** (required for web)
- **blender** (required)
- **rust nightly toolchain + rust-src component** (required):
 - **wasm32-unknown-emscripten** (web)
 - **x86_64-pc-windows-msvc** (if on windows)
 - requires MSVC (from windows C++ build tools)
 - **x86_64-unknown-linux-gnu** (if on linux)
 - requires gcc or clang (should be installed by default)
- **just (>= v1.38.0)** (optional - highly recommended - makes it easy to setup, develop, and release)

You should ensure that these dependencies are in your PATH.

Then you can look at the justfile to find commands you may want to run, or `just --list`

3.2.3 Getting Started with the program post environment installation:

To launch godot with the correct environment run:

```
just godot
```

To launch an editor of your choice (e.g. VSCode) with the correct environment run:

```
just env "code ."
```

(note: you need to pass the command as a string)

To compile the Rust code before launching the game through Godot, run:

```
just rust-debug rust-release
```

Ensure that the export templates are downloaded, which can be done by opening the godot editor, and going to project → export → web, and following the instructions in red at the bottom of the menu. (manage export templates → download and install).

Then you can export the game with `just release-web`, which will create folders in the root of the project at `target/web`, this folder can be deployed as a webroot on any webserver.

The webserver must serve these files through HTTPS, with a valid certificate, along with the following HTTP headers set:

```
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Embedder-Policy: require-corp
Cross-Origin-Resource-Policy: cross-origin
```

```
Permissions-Policy: cross-origin-isolated=(self)
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
```

To ease the burden of deployment, a CI/CD pipeline is available on GitHub to deploy to our current instance at <https://ibm.alexddias.dev>, which is configured to deploy to Netlify using defined secrets in the repository.

3.3. Launching: Provide instructions on creating a user account or logging into the system. Guide users through any first-time setup steps, such as creating admin accounts or configuring user settings.

1. Navigate to the game's URL (<https://ibm.alexddias.dev>) and wait for the game to load.
2. If the browser asks for any permissions, choose **Allow**.
3. As a first time player it is recommended that you navigate to the **options** section and adjust the settings to best suit you.
4. Choose to play the game in the 3D or VR viewport.
5. After playing the game in your desired mode (**VR** or **3D**), you will have the option of saving your score to the leaderboard under the username of your choosing.

See Section 1.1 for more information on how to play, along with the upcoming usage manual.

3.4. Troubleshooting: List common error messages and solutions for resolving them. Explain where users can find logs or diagnostic information to troubleshoot issues.

Users may commonly encounter the following errors as listed below with their causes and solutions:

1. VR Compatibility Issues:

- **Error Message:** Could not start VR
- **Cause:** Attempting to use VR features without a compatible browser or headset.
- **Solution:** Verify that your browser supports WebVR/WebXR and ensure your VR headset is correctly connected and configured.

2. Non-Secure Environment:

- **Error Message:** Error messages indicating missing dependencies
- **Cause:** Loading the website while missing relevant HTTP or HTTPS headers as well as missing the relevant dependencies.
- **Solution:** Ensure all permissions have been granted and all pre-requisites have been installed.

3. General Errors:

- **Error Message:** Errors and warnings are printed within Devtools inside the console for web builds, or within the Godot Output Terminal for native Godot execution.
- **Cause:** Issues related to scripts, nodes, resources, or scene configuration.
- **Solution:** Examine error and warning messages within the Godot Output Terminal or the. Adjust scripts or resources as necessary based on detailed feedback.

4. Game Loading Errors:

- **Error Message:** loading errors displayed in a red message box on-screen.
- **Cause:** Script errors, or resource loading failures.
- **Solution:** Review the specific error message displayed. Further information can be found in the devtools console.

4. Maintenance and Implications

4.1. Provide useful and usable information on how the system can be maintained

- **Regular Content Updates**
 - Update and create new question banks for more topics (e.g. theoretical computer science, cloud computing).
 - Use built-in question bank editing tool for adjusting current questions to be more topical.
- **Software/Hardware Compatibility**
 - Test updates on Quest, different operating systems and web browsers before pushing to production.
 - Expand VR headset compatibility to other headsets such as Valve Index, HTC Vive etc.
- **Performance Optimisation**
 - Monitor VR metrics: Frame rate, Latency to ensure new updates and features do not reduce user performance.
 - Clean redundant data (deprecated topics) from database.
- **User Feedback Integration**
 - In-game bug reporting.
 - Question flagging system for inappropriate or wrong questions.

4.2. Provide useful and usable information on how the system's possible future development can be implemented

- **AI-Driven Question Generation**
 - Adopt generative AI in the generation of questions, use academic papers and textbooks (with permission) as training material.
- **Collaborative Features**
 - Multiplayer mode for collaborative or competitive learning environment.
 - Allow importation of question banks created by other users
- **Cross-Platform Expansion**
 - Allow web version to be accessed through mobile to enhance accessibility among those without normal VR headsets
 - Decrease system requirements through introducing graphics settings to widen user base.
- **Advanced Analytics**
 - Create a individualised user dashboard with focus areas on subjects that need improvement, as well as strong areas for users to reflect and learn from.

4.3. Describe potential ethical and societal impacts of the system in its current and possible future status

- **Current System:**
 - Accessibility Divide:
 - VR hardware costs may exclude low-income users, exacerbating inequities in tech education. To combat this, team has created a 3D version accessible through traditional web, so that any user with a modern computer would be able to access and play the game.
 - Our planned accessibility settings allow for customisable text scaling, colourblind-friendly palette adjustments and control remaps to accommodate users with visual impairments, colour vision deficiencies and motor control deficiencies.
 - Educational Integrity:

- Ensure question banks are fact-checked to avoid spreading misinformation (e.g., outdated coding practices).
- Ensure sourced content for questions are not plagiarised to prevent copyright infringement as well as preserving academic and professional integrity. (Should not be a problem in this case as questions are sourced from IBM SkillsBuild with permission from the company)

- **Future Scenarios:**

- ▶ Ethical Content Moderation:
 - User-generated questions (e.g., on AI ethics) must be moderated to filter harmful or biased content.
 - Question bank tool provides a method for quick editing of questions that violate the educational context of the game.
- ▶ Societal Benefits:
 - Democratising CS education through gamification could inspire underrepresented groups to pursue tech careers.
 - Easier to approach method of learning about technology (video games) provides spark needed for people to start seriously learning about code.
 - Over-reliance on VR learning might reduce hands-on traditional learning, need to reinforce to users that traditional learning is still needed to succeed.